Reproducing SkillOpt on VeruSAGE: An Epoch-1 Comparison of Actor and Optimizer Configurations

August 13, 2026

Abstract

This document summarizes our current reproduction of SkillOpt on VeruSAGE proof-repair tasks. The reproduced system evolves one monolithic skill and does not perform retrieval at inference time. We use a frozen split of 40 training problems, 20 selection problems, and 40 held-out test problems. A first epoch evaluates the initial skill on the selection set, collects 40 training rollouts, updates the skill with an optimizer, and evaluates the candidate on the same selection set. Later epochs reuse the score of the current accepted skill and therefore require only 40 training rollouts and 20 selection-gate rollouts. The candidate is accepted only if its selection accuracy strictly improves. With DeepSeek V4 Flash as the actor, both the original Flash optimizer and the stronger GPT-5.6 Sol optimizer produced candidates that decreased selection accuracy from 6/20 to 4/20. A DeepSeek V4 Pro optimizer candidate was rejected by an evidence audit before live evaluation. In the latest complete epoch, using DeepSeek V4 Pro as the actor and GPT-5.6 Sol as the optimizer improved selection accuracy from 16/20 to 17/20, giving the first accepted candidate in this reproduction. This is an in-loop selection result; the 40-problem held-out test set has not been evaluated.

1 Reproduction Scope

The current SkillOpt reproduction maintains a single textual skill. The whole skill is supplied to the actor for every rollout, and the optimizer revises that skill using actor execution traces. The current version does not contain a runtime retrieval component and therefore does not select task-specific skill clauses or memories. The experiment studied here is consequently a reproduction of iterative global-skill optimization, rather than a retrieval- augmented extension.

SkillOpt separates model use into two roles:

- **Actor:** receives the current skill and executes a VeruSAGE proof-repair task. Its trajectory, verifier outcome, and final candidate are recorded.
- **Optimizer:** analyzes the training trajectories, reflects on successes and failures, merges proposed changes, ranks the changes, and produces the next candidate skill.

The standard SkillOpt configuration uses GPT-5.5 for both roles. Our experiments vary the actor and optimizer independently to determine whether the negative Flash results are associated with the optimizer, the actor trajectories, or both.

2 Data Split and Epoch Workflow

All reported experiments use the same frozen Anvil/IronKV split. Its SHA-256 identifier is

`53059264e5d0458e1fc50a3c1786cbea`
`c6c671aedf56dd71fb32843b24d2c553`.

The split contains 40 training problems, 20 selection problems, and 40 held-out test problems. The selection set is used only for in-loop skill evaluation. Training rollouts are the only task trajectories supplied to the optimizer. The held-out set is not used in the experiments summarized here.

For the first epoch of one continuous SkillOpt run, the actor workload is

$$N_{\text{actor}} = N_{S_0} + N_{\text{train}} + N_{\text{gate}} = 20 + 40 + 20 = 80 \text{ task rollouts.}$$

The initial S_0 selection evaluation is a one-time initialization cost. Once an accepted skill and its selection score have been established, each later epoch requires only

$$N_{\text{later epoch}} = N_{\text{train}} + N_{\text{gate}} = 40 + 20 = 60 \text{ task rollouts.}$$

Consequently, a continuous run of E epochs requires

$$N_{\text{actor}}(E) = 20 + 60E,$$

excluding any one-time held-out test evaluation. For example, two epochs use 140 actor rollouts and four epochs use 260. An epoch would return to 80 only if it were launched as a separate experiment that deliberately reruns the 20-problem baseline, rather than continuing from the stored accepted score.

The workflow is shown in Table 1. Here, S_0 is the initial accepted skill, and S_1 is the candidate produced by the optimizer.

Step	Stage	Actor rollouts	Purpose
1	Initial selection evaluation	20	Measure S_0 on the fixed selection set.
2	Training rollout	40	Generate success and failure trajectories using S_0.
3	Skill optimization	0	Reflect, merge, rank, and construct candidate S_1.
4	Candidate selection gate	20	Measure S_1 on the same selection problems.
5	Gate decision	0	Accept S_1 only if its strict solved rate exceeds S_0.

Table 1: One-epoch SkillOpt workflow used in the reproduction.

For an optimizer replay, stored training trajectories can be reused. Such a replay requires no new training rollouts; only the candidate's 20-problem gate is rerun. We distinguish these replays from complete epochs throughout the results.

3 Experimental Configurations

Four actor–optimizer configurations have been investigated:

1. **Flash/Flash (F/F):** a complete epoch using DeepSeek V4 Flash as both actor and optimizer.

2. **Flash/Pro (F/P):** offline reanalysis of the stored Flash training trajectories using DeepSeek V4 Pro as the optimizer. The final candidate was rejected before the live gate because deterministic and manual audits identified incorrect evidence attribution.

3. **Flash/Sol (F/S):** native SkillOpt reflection, merge, and ranking over the same stored Flash trajectories using local Codex GPT-5.6 Sol as the optimizer, followed by a fresh 20-problem Flash actor gate.

4. **Pro/Sol (P/S):** a complete epoch using DeepSeek V4 Pro as the actor through the Codex CLI native Responses harness and local Codex GPT-5.6 Sol as the optimizer.

The initial skill is the same 838-byte artifact in all configurations. The Flash replay experiments inherit the S_0 and training measurements from the complete F/F epoch instead of regenerating them.

ID	Actor	Optimizer	S_0	Train	Candidate	Δ	Gain/Loss	Skill bytes	Decision
F/F	DS V4 Flash	DS V4 Flash	6/20	8/40	4/20	−2	0/2	10,322	Reject
F/P	DS V4 Flash	DS V4 Pro	6/20†	8/40†	–	–	–	1,646	Pre-gate reject
F/S	DS V4 Flash	GPT-5.6 Sol	6/20†	8/40†	4/20	−2	0/2	3,490	Reject
P/S	DS V4 Pro	GPT-5.6 Sol	16/20	35/40	**17/20**	**+1**	**1/0**	2,932	**Accept**

Notes. Δ is the change in solved problems on the 20-problem selection set. Gain/Loss reports fail-to-pass and pass-to-fail transitions, respectively. † indicates that the Flash S_0 result and 40 training trajectories were reused from the F/F epoch rather than rerun. No row includes held-out test evaluation.

Table 2: Selection-gate results for the SkillOpt reproduction.

Actor	Optimizer	Tasks	Requests	Input (cached)	Output	Actor cost	Cost/task	Optimizer cost
DS V4 Flash	DS V4 Flash	80	4,184	35.528M (27.766M)	14.403M	\$5.1971	\$0.0650	\$0.0351
DS V4 Flash	DS V4 Pro	0	0	–	–	\$0	–	\$0.0581
DS V4 Flash	GPT-5.6 Sol	20	1,627	12.925M (9.559M)	5.447M	\$2.0230	\$0.1011	\$0 local
DS V4 Pro	GPT-5.6 Sol	80	3,559	197.107M (194.614M)	2.986M	\$4.3879	\$0.0548	\$0 local

Notes. Parentheses in “Input (cached)” report cache-hit input tokens. Cost/task is actor cost divided by the number of newly executed actor tasks; the Flash/Sol value covers a 20-task candidate gate rather than a full epoch. The Flash/Pro row is optimizer-only reanalysis of stored Flash trajectories, so it has no new actor tasks. Optimizer costs use model-specific prices recorded at experiment time. Flash/Flash uses the Flash rates: \$0.0028/M cache-hit input, \$0.14/M cache-miss input, and \$0.28/M output. Flash/Pro uses the Pro rates: \$0.003625/M cache-hit input, \$0.435/M cache-miss input, and \$0.87/M output. Because the optimizer logs do not retain a prompt cache split, their DeepSeek prompt tokens are conservatively treated as cache misses. The displayed Flash/Pro value is the final Pro-v2 analysis; including the earlier rejected Pro-v1 analysis gives \$0.1126. F/S and P/S used local Codex quota for the GPT-5.6 Sol optimizer and therefore had zero metered optimizer cash cost. At the recorded Sol API rates and treating all optimizer input as uncached, their API-equivalent costs would be approximately \$1.5671 and \$5.7069, respectively, before any long-context uplift. P/S actor cost excludes Pro-native preflights and a stopped wrong-reasoning startup. Including those bring-up calls yields approximately \$4.7360 for that Pro launch.

Table 3: Measured actor usage and optimizer usage. Dollar values are estimated from the provider usage ledgers.

4 Primary Results

Table 2 gives the raw solved counts. “Gain/Loss” is a paired comparison on the fixed selection tasks: gain is fail-to-pass and loss is pass-to-fail. The F/P row deliberately contains no candidate score because the candidate never entered a live selection gate. Reporting it as 4/20 would incorrectly convert a pre-gate audit rejection into an evaluated result.

The Flash actor produced the same negative gate result with two different evaluated optimizers. The F/F candidate introduced zero new solves and lost two baseline solves. Replacing the optimizer with GPT-5.6 Sol produced a shorter, audit-clean candidate, but the paired selection outcome remained exactly zero gains and two losses. The Pro optimizer analysis cannot be included in this numerical comparison because its candidate was stopped before target evaluation.

The P/S epoch produced the first accepted update. The initial skill solved 16/20 selection tasks, the training rollout solved 35/40 tasks, and the candidate solved 17/20 selection tasks. The paired result was one gain, zero losses, 16 retained successes, and three retained failures. SkillOpt therefore accepted the candidate because 17/20 strictly exceeded 16/20.

5 Usage and Cost

Table 3 separates newly executed actor work from optimizer work. This distinction is important for replay experiments: F/P ran no actor tasks, and F/S reused all 40 training trajectories and paid only for a new candidate gate. The P/S actor ledger includes the full initial evaluation, training rollout, and candidate gate.

In these measured runs, the Pro/Sol actor cost less per task than the Flash/Flash actor (\$0.0548 versus \$0.0650), but this does not mean that Pro has lower token prices. The Pro/Sol epoch used substantially more input tokens than Flash/Flash, but 98.7% of its input was served from the DeepSeek prompt cache and it produced only 2.986M output tokens. By comparison, Flash/Flash had a 78.2% input cache rate and produced 14.403M output tokens. Output alone cost approximately \$4.0327 in Flash/Flash, versus \$2.5980 in Pro/Sol. The cache-hit, cache-miss, and output contributions were therefore \$0.0777, \$1.0866, and \$4.0327 for Flash/Flash, compared with \$0.7055, \$1.0844, and \$2.5980 for Pro/Sol. This token-shape difference outweighs Pro's higher unit prices, yielding formal actor costs of \$5.1971 and \$4.3879, respectively. This is a measured harness outcome, not evidence that Pro is intrinsically cheaper. The local GPT-5.6 Sol optimizer incurred no metered API cash cost in either Sol experiment.

6 Interpretation

Optimizer strength alone did not rescue the Flash actor. The evaluated F/F and F/S candidates both changed selection performance from 6/20 to 4/20, with identical paired transitions of zero gains and two losses. The stronger Sol optimizer improved skill compactness and passed the skill contract audit, but this did not translate into a better Flash actor gate.

The Pro actor generated more useful optimization evidence. In the P/S epoch, the Pro actor solved 35/40 training tasks and the optimizer produced an accepted candidate with a paired +1/0 selection change. This is consistent with the hypothesis that higher-quality actor trajectories can provide more useful evidence for skill optimization. It is not, by itself, a causal isolation of actor capability because the Pro experiment also used a different, Codex-native target harness and a higher initial S_0 score.

The gate prevented harmful updates. Both evaluated Flash candidates were rejected and the accepted skill remained S_0. The Pro candidate was accepted only after a strict improvement on the same fixed selection tasks. This demonstrates the intended mechanical role of the SkillOpt gate even though it does not establish population-level gains.

The Pro-optimizer result is qualitative, not a gate score. The F/P candidate was compact, but its evidence audit found incorrect causal attribution of a verifier failure. Stopping it before the live gate was an integrity decision. It should not be grouped numerically with the two 4/20 Flash gate results.

7 Validity and Limitations

The evidence has four boundaries. First, every selection comparison contains only 20 tasks, so one task corresponds to five percentage points and sampling uncertainty is large. Second, the experiments are single runs without an A/A estimate of actor stochasticity. Third, the old Flash and new Pro experiments share the frozen split and gate semantics but not an identical target harness; the latest positive result cannot be attributed solely to changing the actor model. Fourth, the selection set is reused for in-loop acceptance and is not a held-out generalization set. No claim of improved held-out solved rate or token efficiency is made here.

All 80 final task results in the P/S epoch completed with 77 full V2 traces, three timeout-preserved V1 traces, zero invalid V0 results, zero provider errors, and unchanged source inputs. The 40 held-out test tasks were not run in any experiment summarized by the primary table.

8 Conclusion

The reproduction confirms the core mechanics of single-skill SkillOpt on a 40/20/40 VeruSAGE split. With DeepSeek V4 Flash as actor, changing the optimizer from Flash to GPT-5.6 Sol did not change the negative selection outcome: both evaluated candidates moved from 6/20 to 4/20 and were rejected. The DeepSeek V4 Pro optimizer candidate was rejected before live evaluation and therefore has no selection score. The latest complete P/S epoch, using DeepSeek V4 Pro as actor and GPT-5.6 Sol as optimizer, moved from 16/20 to 17/20 with one paired gain and no paired loss. This candidate passed the first gate and became the first accepted evolved skill in the current reproduction. The result is evidence of a successful in-loop update, not held-out effectiveness evidence.